
BODIOS

iOS build roadmap and agent prompts

React Native Community CLI | TypeScript | SQLite

Prepared for Sam Karimpour
PDF edition - 26 September 2026

How to use this document

Attach this PDF and both design images to your coding agent. Start with the Master prompt and Stage 1. Run and test each stage before moving to the next.

Includes the full iOS roadmap, copy-paste agent prompts, Mac setup, security and optional accounts, plus TestFlight and App Store preparation. The design images and actual exercise videos are separate assets.

Scope: guest/offline app first. Accounts and cloud sync remain optional. If accounts are needed at launch, complete Stages 8A and 8B before releasing that feature.

Updated for iOS development on Sam's Apple Silicon MacBook. React Native Community CLI and TypeScript remain the stack. This edition replaces the Android-specific build instructions; reuse existing app code and preserve the Android native project if present.

Mac setup and a simple startup command

The agent needs access to this project and a terminal on your Mac. A remote Linux agent cannot run your Mac's Xcode Simulator. Clone or copy the complete existing repository first, including package manifests and lockfiles; do not copy node_modules from Windows. Install dependencies fresh using the existing package manager.

Use full Xcode, its Command Line Tools, and an installed iOS Simulator runtime. Command Line Tools alone are insufficient. The agent can inspect versions and install project packages, but Apple account sign-in, Xcode downloads, license dialogs, device trust and signing-team choices may require you. Do not enter Apple passwords into agent prompts. A paid developer membership is not needed for Simulator development; App Store/TestFlight distribution requires the appropriate membership and signing setup. [I1, I3]

Setup checks: `sw_vers`; `uname -m`; `xcodebuild -version`; `xcode-select -p`; `xcrun simctl list devices` available; `node --version`; `ruby --version`; `bundle --version`. Check versions against the existing project's requirements before upgrading. Use native Apple Silicon tools where supported; do not apply blanket Rosetta or simulator-arm64 exclusions as a default fix.

Respect npm/yarn/pnpm lockfiles. For npm, ensure package scripts include start for Metro and ios for the local React Native CLI. Typical development uses `npm start` in one terminal and `npm run ios` in another. Select the installed simulator by its actual name/UDID using the local CLI's supported flags. These are project commands, not global React Native installations. [I2]

Add a custom npm run `dev:ios` script (or equivalent for the existing package manager) that checks prerequisites, selects/boots an available simulator, starts or reuses this project's Metro server, and launches the app. Reuse compatible local CLI tools. Handle occupied ports without killing unrelated processes, propagate errors, and clean up child processes it started. Document a two-terminal fallback. This shortcut is to be implemented and tested; it is not assumed to exist already. First-time setup, Pods after native changes, Apple dialogs and release signing are separate tasks.

For any local backend, configure an environment-specific API base URL. Android emulator host aliases such as 10.0.2.2 do not belong in iOS configuration. A physical iPhone needs a reachable server address; localhost on that phone is the phone itself. Keep production endpoints HTTPS and keep development-only network exceptions out of Release builds.

Keep existing Android code if present, but prioritize iOS builds and testing. Local Android SQLite files do not automatically move to an iPhone; reuse the schema and add explicit export/import or future cloud sync if existing device data must be transferred.

Product direction

Build a React Native CLI gym tracker for iOS with short exercise demonstrations, custom routines, reliable set logging, and progress history. Use the charcoal and electric-lime Bodios mockups as visual direction. The images are static concepts; their sample numbers and generated athlete imagery are not production data or verified exercise instruction.

Target iPhone first. Use the iPhone Simulator included with Xcode, then test on a physical iPhone. Detect installed simulator models and iOS runtimes; do not assume a particular model exists. Use

responsive layouts, safe areas for the notch/Dynamic Island and home indicator, keyboard avoidance, and scalable text. Do not hardcode device pixel dimensions. Dedicated iPad and macOS app support are separate scope; owning a Mac does not turn this into a Mac desktop app.

Recommended first release

Include guest use, searchable exercise library, exercise detail with video and written instructions, routine builder, active workout, rest timer, resumable workout drafts, workout summary, history, progress, and settings. Start with about 20 exercises across the major movement categories. Begin implementation with one complete example: dumbbell curl.

Defer accounts/cloud sync, AI coaching, camera form analysis, social feeds, subscriptions, wearables, nutrition, and 3D bodies until the core workout flow works. These are future options, not unfinished MVP requirements.

Video recommendation and content workflow

Use short MP4 clips of approximately 8-15 seconds showing two or three controlled repetitions. Start with 720p, H.264, and a consistent background and framing; inspect the result on an iPhone before choosing final encoding settings. Keep the whole movement visible. A front/three-quarter angle is a starting point; some movements need a side angle too.

For the first prototype, obtain a few clips you recorded or have explicit permission to use. For a larger catalogue, evaluate a licensed exercise-video provider. Do not assume a searchable exercise API includes video rights, or invent media URLs. Record source, permitted usage, attribution requirements, and review status for each asset. Do not download random social-media clips into the app.

Ask a qualified trainer to review exercise names, equipment, demonstration technique, instructions, and common mistakes before publication. Generated UI athlete posters only communicate placement and mood; replace them with exercise-specific reviewed thumbnails and recordings. The image-generation tool creates the mockup, not exercise video files.

Store content metadata separately from the video files. For development, bundle a few approved clips using a static asset map. For a larger catalogue, put approved videos and posters in object storage behind a CDN, using HTTPS. Start with no paid provider dependency; the owner supplies approved media. Display a clear unavailable state when a clip is missing.

Player behavior: poster first, tap to play, muted initially, loop, pause, replay, seek, fullscreen, and 0.5x/1x speed. Do not autoplay videos in scrolling lists. Pause when leaving the screen or backgrounding the app. Keep only one demo playing. Preserve the full movement with contain fitting rather than cropping limbs. Show loading, retry, and unavailable states. Written instructions and logging remain usable when video fails.

Offline promise for MVP: routines, instructions, workout logging, and history work without internet; bundled demos do too. Remote demos require a connection unless explicitly downloaded. Temporary playback cache is not a guarantee of offline availability. Add explicit download/remove controls and bounded storage later if needed.

Architecture

Use React Native Community CLI, TypeScript, React Navigation, react-native-video, and SQLite through @op-engineering/op-sqlite. Use native-stack and bottom-tabs navigators, with the compatible react-native-screens and react-native-safe-area-context dependencies. Do not introduce Expo, Expo Router, expo-video, expo-sqlite, Expo Go, or Expo build commands into this plan.

SQLite is the database engine; OP-SQLite is the library that lets React Native access it. Save a persistent database in the app's private data directory for routines, sets, sessions, and settings. No database server is needed for local logging. Closing the app retains this data; loss of the device or clearing app data requires a backup to recover it. Video files remain separate from the database, which stores their references.

For an existing project: inspect dependencies, React Native version, architecture, navigation and iOS build configuration first. Preserve working code and use compatible library versions recorded in the lockfile. Do not recreate or upgrade the whole app just to match a mockup. If earlier implementation followed the Expo draft, replace the affected setup, navigation, database adapter and video component while preserving features and saved data; do not delete a database as a migration shortcut.

For video, select a stable release compatible with the installed React Native version and iOS deployment target. Read the documentation for that exact major version; do not silently install prereleases or mix player APIs between versions. Use React Native AppState and navigation focus events for lifecycle handling. Verify native iOS dependencies for both Apple Silicon Simulator and physical-device builds.

Build with the project's Xcode and CocoaPods workflow. Check the macOS/Xcode/React Native compatibility matrix, Node, Watchman, Ruby/Bundler, CocoaPods, and installed iOS runtimes. Use the existing Gemfile and lockfiles. Run `bundle install` and `bundle exec pod install` from their appropriate project directories when needed. Native dependency changes require updating Pods and rebuilding iOS; Metro refresh alone is insufficient. Open the generated `.xcworkspace` when using CocoaPods. Test SQLite persistence and video on Simulator and a real iPhone.

Keep screens, reusable UI components, domain logic, database repositories, exercise content, and media handling separate. Database-backed state is the source of truth for saved workouts. A small React context or the existing state library is sufficient for transient UI state. Never connect a mobile client directly to a private MongoDB database or embed service secrets.

Suggested folders: `App.tsx` (entry UI), `src/navigation/`, `src/components/`, `src/features/exercises/`, `src/features/routines/`, `src/features/workouts/`, `src/features/progress/`, `src/data/`, `src/db/`, `src/theme/`, `assets/exercises/`, and the CLI-generated `ios/` native project. Adapt names to the existing repository.

Implementation references (check the installed versions before using APIs):

- [React Native Community CLI setup](#): create and run an app with native projects.
- [OP-SQLite installation](#): SQLite access for React Native, including iOS.
- [React Native Video v6](#): Video component, local files, streaming, and iOS support.
- [React Native Video v7 installation](#): check the release label and requirements before choosing a major version.
- [React Navigation setup](#): follow the Community CLI installation path and matching iOS configuration.
- [React Native iOS Simulator](#): detect and launch the installed simulator.

Screens and behavior

Screen	Required behavior
Welcome	Short introduction, guest entry, unit preference; no login wall
Home	Resume active workout, saved routines, actual weekly activity, useful empty state
Workouts	Saved routines and exercise library; search and muscle/equipment filters
Exercise detail	Correct demo/poster, muscle and equipment tags, instructions, mistakes, add-to-routine action
Routine builder	Name, ordered exercises, target sets/ reps/ rest, add/remove/reorder, persist
Active workout	Current exercise, watch demo, previous performance, editable set rows, complete/undo, rest timer, finish
Summary	Actual completed work, duration, notes; no invented calorie burn
Progress/history	Session list, detail, date filter, workout count, per-exercise weight/ reps trends
Settings	kg/lb, default rest duration, media preference, export data, explicit reset confirmation

Bottom tabs: Home, Workouts, Progress, Profile. Exercise detail and workout flow use stack navigation. Support iOS back buttons, swipe-back and modal dismissal without losing saved workout drafts. Fullscreen video needs an obvious exit. Discard is a separate confirmed action. Adapt the existing charcoal/lime mockups to iOS navigation and safe areas; do not copy iOS system bars into the interface.

Design tokens: background #0B0D0C, surface #171A18, accent #BEFA57, main text #F5F7F4, secondary text #A8AFA9. Use 8-point spacing, approximately 16-20 logical units of screen padding, 16-24 card radii, and at least 48-point touch targets as the Bodios product target. Use dark text on lime buttons. Support text scaling, screen-reader labels, and non-color completion indicators. Do not draw fake status bars or phone frames inside the real app.

Data model and metric rules

Entity	Fields
Exercise	id, name, aliases, primaryMuscles, secondaryMuscles, equipment, instructions[], mistakes[], trackingType, weightConvention, mediald
ExerciseMedia	id, exerciseld, localAssetKey or remoteUrl, poster, durationSeconds, source, permissionNotes, attribution, reviewStatus
Routine	id, name, createdAt, updatedAt
RoutineExercise	id, routineld, exerciseld, position, targetSets, targetReps, targetDurationSeconds, restSeconds
WorkoutSession	id, routineld nullable, status, startedAt, endedAt, trainingLocalDate, timezone, pausedDurationMs, notes
SessionExercise	id, sessionId, exerciseld, position, nameSnapshot, trackingTypeSnapshot, weightConventionSnapshot
WorkoutSet	id, sessionExerciseld, position, type warmup/work, weightKg nullable, reps nullable, durationSeconds nullable, completedAt nullable
ActiveWorkoutState	sessionId, currentSessionExerciseld, restEndsAt nullable, restPausedRemainingMs nullable, pausedAt nullable
Settings	preferredUnit, defaultRestSeconds, mediaPreferences

Use stable IDs, foreign keys, migrations, parameterized SQL and transactions for related writes. Snapshot session details so changing a routine later does not rewrite workout history. Persist the in-progress session after each meaningful edit. Prevent duplicate finish and complete-set actions.

Store weights in canonical kg and convert for display. Label dumbbell exercises "weight per dumbbell"; label barbell exercises "total including bar." Do not silently change conventions. Reps are positive integers when completing a repetition set; weights allow decimal values, and zero can be valid for bodyweight or unweighted movements. Timed exercises use duration instead of fabricated repetitions.

Workout count = finished sessions containing at least one completed working set. Best weight = highest logged completed working-set load for that exact exercise and load convention; show its reps and date. Do not label this as a tested one-rep max. Count activity days using the stored training local date. Start with weekly consistency rather than encouraging a daily workout streak. Empty history means empty-state UI, not sample achievements.

If adding training volume later, document whether a dumbbell load counts one or both implements; do not compare incompatible exercises or assisted/bodyweight movements using a misleading total. Progress charts initially show per-exercise best logged weight and completed workout count.

Rest timer: persist an end timestamp, calculate remaining time from the clock, and recompute on foreground. Do not depend on a JavaScript interval running in the background. Pause/resume must persist remaining duration explicitly. A session that crosses midnight keeps its recorded training date. Background alarm delivery is outside MVP unless notifications are deliberately implemented and tested.

Roadmap and copy-paste prompts

Paste the master prompt once, then send the numbered prompts one at a time. Ask the agent to finish and verify each stage before the next. Estimates below are planning ranges for focused development sessions, not delivery guarantees; obtaining and reviewing media can take longer than coding.

Master prompt

COPY-PASTE AGENT PROMPT

You are implementing Bodios, my iPhone gym tracker, using React Native Community CLI and TypeScript on my Apple Silicon MacBook. I am not using Expo. Read the attached Bodios-iOS-Roadmap.pdf and inspect my existing repository before changing it. Reuse implemented features and saved-data schemas. Detect Xcode, active developer directory, installed iOS runtimes, and available simulators. Report actual device/runtime tests; never claim Simulator or physical-device tests you could not run. If ios/ is missing, restore or generate compatible native iOS files without replacing the whole app or deleting android/.

Use the attached mockups as visual direction: charcoal surfaces, electric lime buttons, bold readable typography, workout photography and rounded cards. Implement real native UI and data-driven screens. The mockup is not a functional app, its demo numbers are not my data, and its generated athlete images are not verified exercise instruction.

Preserve working code and repository instructions. Use React Navigation for tabs and native-stack navigation, react-native-video for demonstrations, and SQLite through @op-engineering/op-sqlite for persistent local data. Do not introduce Expo packages or Expo build commands. Adapt to the installed React Native version and iOS configuration. If earlier code followed the Expo draft, migrate only affected parts and preserve saved data. Check official docs against installed versions and select stable compatible releases before using APIs. Do not replace the project or upgrade all packages without a concrete need.

Work only on the numbered stage I request. Complete the implementation, run relevant checks and report changes, commands, results, remaining blockers and a brief manual verification flow. Explain key choices plainly because I am learning React Native. Keep business logic separate from screens. No invented video URLs, no fake finished integrations, no embedded secrets. Missing videos must have usable fallback UI. Keep genuine user history separate from development fixtures. Build the offline workout flow before adding cloud accounts. Follow the Security requirements appendix in this PDF from the start; if accounts are enabled, complete Stage 8B and its security acceptance tests before releasing them.

1 - Inspect and boot (1-2 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 1. Inspect repository instructions, package manager, app entry points, routing, installed versions, existing backend, and ios/ configuration. Reuse working code. If no app exists, scaffold a TypeScript React Native Community CLI project with compatible stable tooling. Perform the Mac setup checks in this document, install compatible project dependencies, and configure CocoaPods with existing Gemfile/lockfiles. Do not scaffold Expo. Configure React Navigation native-stack and four bottom tabs with required native dependencies. Add the Bodios theme, reusable Button/Card/Text/Input components, iOS safe areas and routes for exercise detail and active workout. Preserve drafts on swipe-back and modal dismissal. Build and launch on an available iPhone Simulator, fixing build errors. Verify navigation and a genuine empty-state Home screen. Add and test the documented startup scripts, including `npm run dev:ios` if npm is the package manager. If blocked by Xcode installation or account interaction, state the exact manual step and complete all unblocked work. Do not fabricate statistics or successful tests.

2 - Exercises and local data (2-3 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 2 using the roadmap data model. Install a compatible `@op-engineering/op-sqlite` version and rebuild iOS. Use a persistent database in app-private storage, with versioned SQLite migrations and repositories. Keep database calls out of screen components; use parameterized statements and transactions for related writes. Create a starter catalogue of approximately 20 exercises covering chest, back, shoulders, arms, legs and core, including dumbbell curl and hammer curl as distinct variants. Add aliases, muscles, equipment, tracking type, weight convention and draft instruction fields. Flag content requiring trainer review. No invented media links. Build searchable/filterable exercise lists and exercise detail. Render approved assets if available, otherwise a clearly labelled missing-demo state. Keep the screen usable without internet. Verify migration startup, search/filter combination, empty search and data persistence after restart on iOS. JavaScript mocks alone do not verify native persistence.

3 - Demonstration player (1-2 sessions plus media preparation)

COPY-PASTE AGENT PROMPT

Implement Stage 3. Add a reusable `ExerciseVideoPlayer` using a stable react-native-video release compatible with this React Native project; use the matching major-version API and docs. Do not use Expo video APIs or silently install a beta. Update CocoaPods as required and rebuild iOS after native dependency changes. Verify fullscreen, mute behavior, audio interruptions and app backgrounding on iOS; do not add background audio capabilities just to keep a demo running. Resolve sources through exercise-specific metadata and a static map for bundled require assets. Accept approved local MP4s and HTTPS media sources. Show poster, tap-to-play, muted initial state, looping, play/pause, replay/seek, fullscreen, and 0.5x/1x speed using supported APIs. Preserve full-movement framing; for v6 use `resizeMode="contain"`. Pause using React Navigation focus events and React Native AppState; release resources correctly. Only one player may play. Handle loading, buffering, unavailable source and retry without blocking set logging. Do not autoplay exercise-list videos. Explain how I can supply and replace the dumbbell-curl clip, including exact folder and metadata entry. If no approved clip exists, report that playback still needs a real asset; never treat a generic clip as a curl demonstration. Verify on iOS with an actual supplied clip if available.

4 - Routine builder (1-2 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 4. Let me create, rename and delete a routine; search/add exercises; edit target sets, reps or duration and rest; remove and reorder exercises with accessible up/down controls. Save everything locally. Add to workout on exercise detail should open a routine picker or let me create a routine. Keep target prescriptions separate from logged results. Validate inputs and prevent starting an empty routine. Verify a routine survives app restart and that changing it cannot alter previous sessions.

5 - Active workout and recovery (3-4 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 5. Starting a routine creates an in-progress session snapshot. Show current exercise, a Watch demo action, previous logged performance, editable sets and a clear load convention. Allow adding/removing sets, completing/undoing a set, navigating exercises and recording notes. Persist edits immediately with appropriate transactions and protect against double taps. Keep actual weight/reps blank until entered or explicitly copied from previous performance. Add the timestamp-based rest timer with pause/resume, skip and duration adjustment. Restore the workout and timer after backgrounding and force-closing. Leaving the screen keeps the draft; discarding requires explicit confirmation. Finish exactly once and show a summary computed from completed sets. Verify process restart, unfinished sets, invalid inputs, timed/bodyweight exercises and duplicate finish attempts.

6 - Progress and settings (2-3 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 6. Add history list and session detail, editable notes and explicit delete confirmation. Calculate finished workout count and per-exercise best logged working-set weight according to the roadmap; show reps and date with the best load. Add week/month date filters and charts with accessible text equivalents. Derive all metrics from real local records and recompute after deletion. Build Home from saved routines and actual activity, with Resume workout when applicable. Add kg/lb conversion, default rest duration, data export through the platform share flow, and confirmed local-data reset. Conversion must not change canonical stored weights. Do not invent calories, recovery readiness, or one-rep max results. Verify empty history, month/timezone boundaries, conversion and export contents.

7 - iOS polish and reliable beta (2-3 sessions)

COPY-PASTE AGENT PROMPT

Implement Stage 7. Audit small and large iPhone Simulator layouts using installed runtimes, and report exact models and OS versions. Check VoiceOver labels, reading order, larger text, sufficient contrast, 48-point product touch targets, Reduce Motion, swipe-back, modal dismissal, fullscreen exit, keyboard overlap and home-indicator safe areas. Test offline logging, missing/failed videos, app backgrounding, force quit/relaunch, long lists, double taps and history deletion. Check video audio interruptions and foreground recovery without enabling unnecessary background audio.

Run focused tests for persistence recovery, timer calculations, metrics and unit conversion; run the project's typecheck/lint. Verify actual native SQLite and video behavior rather than only JavaScript mocks. Build an iOS Release configuration and verify it runs without Metro. Prepare the

Xcode archive when tooling and signing permit; distinguish a Simulator build from a distributable signed device build. Never claim an .ipa can be installed freely like an iOS APK. Test on a physical iPhone before calling the beta ready. Record signing or device-access blockers honestly. Update README with exact setup/run/test commands, asset replacement instructions and limitations. Stage 9 handles TestFlight and App Store preparation.

8A - Optional accounts and sync: planning

COPY-PASTE AGENT PROMPT

Plan Stage 8 without implementing or provisioning paid services yet. Inspect whether my existing Node/Express backend can support optional accounts and backup. Propose authenticated APIs, server-side ownership checks, secure token storage, versioned records, queued offline writes, idempotent sync, delete tombstones and an explicit conflict policy. Keep SQLite functional offline. Keep private database credentials on the server. MongoDB Atlas behind a Node/Express API remains an optional cloud choice; changing from Android to iOS does not require changing databases. Inspect the existing backend before selecting or replacing its database. Provide API contracts, migration approach, costs to verify and acceptance tests before implementation. Also outline an explicit demo-download manager if offline videos are needed. Do not add AI coaching, subscriptions or social features unless requested.

Final acceptance checklist

- Open app > find Dumbbell Curl > watch the correct supplied demo > create routine > start > log sets > finish > see accurate history.
- Close and reopen the app mid-workout: entered sets and rest state recover.
- No internet: routine, instructions and logging still work; unavailable remote video is honestly labelled.
- iOS back navigation/fullscreen/keyboard and larger fonts do not hide key controls.
- Video completion does not mark an exercise or set complete.
- Failed/double-tapped writes do not duplicate sets or sessions.
- History contains no mockup numbers or invented achievements.
- Approved movement-specific videos replace generated UI posters before release.
- A real iPhone device beta check complements emulator testing.

What to give your coding agent

Attach this file, the Bodios UI images, the repository, and any approved demo clips. Start with the master prompt and Stage 1. The agent should detect an available iPhone Simulator during setup. Attach your existing project, not just the design images. Start with the Master prompt and Stage 1; skip already-completed features only after checking them on iOS. The critical first milestone is one complete, recoverable workout with a working dumbbell-curl demonstration.

Security requirements and account setup

This edition adds explicit requirements to the original roadmap. Stages 1-7 still deliver a guest/offline app. Stage 8A plans accounts and sync; Stage 8B below implements them only when requested. If login is required at launch, complete both before release. SQLite alone does not provide online registration, password recovery, or cross-device identity.

The following items are implementation requirements, not evidence that an app has been audited. The coding agent must show working code and test results; a prompt cannot guarantee security.

A. Baseline for every version

- Keep database credentials, signing passwords and privileged API keys out of the app bundle, repository and logs. Mobile build environment variables are not a secret vault. Use HTTPS for production APIs and remote media, with normal certificate validation; development-only exceptions must not ship. [S1]
- Use parameterized SQLite statements and validate field types, lengths and ranges. Keep local data in app-private storage; do not claim plain SQLite is encrypted. Assess backup/export exposure and whether encrypted storage is needed for the actual data collected.
- Request only permissions used by implemented features. Avoid recording workout details in analytics or crash logs. Make exports deliberate and remove temporary export files after use where supported. Keep dependency versions tracked and address relevant security advisories.
- Preserve data across database migrations. Confirm before destructive reset. Store only information needed for the gym tracker; do not add unrelated health information fields.

B. Accounts and sessions, when enabled

Choose one authentication approach: an established identity service, or the existing Node/Express backend with mature authentication libraries. Inspect the backend first. A managed service still needs correct application authorization. Do not invent password encryption or authentication protocols.

Implement registration, email verification, login, logout, forgotten-password flow and account deletion. If managing passwords on the server, store salted adaptive password hashes using a maintained implementation such as Argon2id, never plaintext or reversible encryption. Rate-limit login and registration; avoid responses that disclose whether an email is registered. Require recent authentication for sensitive account changes. [S2]

Password-reset tokens must be unpredictable, single-use and expire. Return a consistent reset-request response, limit abuse, and invalidate applicable sessions after reset. Use an actual configured email delivery service; do not claim verification or reset works when email is only printed to logs. [S3]

Persist session credentials using a maintained native secure-storage integration backed by iOS Keychain. Do not put refresh tokens or passwords in ordinary SQLite, AsyncStorage, exported files or diagnostic logs. If external sign-in is added, use the provider's supported native flow with PKCE where applicable. [S1]

Define session expiry, refresh and revocation. Validate tokens on the server, including signature and relevant issuer/audience/expiry checks for JWTs. Clear local credentials on logout and revoke the refresh session. Document any remaining access-token lifetime. If refresh tokens are issued, implement rotation and replay handling with the chosen provider or library.

C. User data ownership and cloud sync

Derive the acting user from the validated server session. Check ownership for every routine, workout, set, export and delete operation, including bulk sync. Deny unauthorized access by default. Never trust a request body's `userId` or a hidden screen as an access control. Allowlist writable fields so clients cannot assign ownership or administrative roles. [S4]

When accounts are added, extend local and server records with an owner identity, version and deletion state. Partition cached data by account. Never show Account A's workouts after signing in as Account B. Ask before importing guest workouts into an account; make this import idempotent. On logout, stop pending sync and prevent queued writes from being reassigned to another user. Explain what happens to unsynced work before deleting it.

A deleted account must not return through stale offline sync. Revoke sessions and reject its queued requests; propagate tombstones and define backup-retention behavior. Keep administrator/content-management privileges server-controlled. Validate inputs server-side even when the mobile form already checks them.

8B - Optional account implementation prompt

Use this prompt after reviewing Stage 8A's architecture and choosing an authentication provider or backend. Provider credentials and service configuration must be supplied through appropriate local/server configuration, never pasted into source code.

COPY-PASTE AGENT PROMPT

Implement Stage 8B using the reviewed Stage 8A design and the Security requirements appendix. Preserve the React Native CLI app and offline functionality. Build working registration, email verification, login, logout, password recovery and account deletion. Use the selected identity service or established backend libraries. Configure real email delivery and report any missing credentials as a blocker, not a finished integration.

Implement the appendix's credential storage, session lifecycle, request validation, rate limits and server-enforced ownership controls. Add account-scoped local storage and sync, deliberate guest-data import, and safe handling of logout, account switching and deletion. Keep private server credentials out of mobile code. Reuse the existing backend where appropriate; do not connect the app directly to a private MongoDB database.

Add meaningful integration tests using two independent users. Prove that one cannot read, update, delete, export or sync the other's records by changing identifiers or ownership fields. Test expired credentials, revoked refresh sessions, repeated reset-token use, rate limits and deletion followed by an offline sync attempt. Verify Account A's local data is hidden after switching to Account B. Inspect the release configuration and logs for secrets and development-only bypasses.

Report which checks passed, which could not run and what remains unresolved. Do not describe the app as secure solely because authentication works. Keep the account feature out of production until the relevant tests and service configuration are complete.

D. Security acceptance before release

- Guest version: native persistence works, SQL inputs are parameterized, exports are deliberate, and logs/build artifacts contain no privileged secrets.
- Account version: two-user ownership tests pass across individual and bulk endpoints. Unauthenticated requests fail and client-supplied owner fields cannot change access.

-
- Account version: verification/reset, expiry, logout, account switching, guest import and account deletion have been exercised against real configured services.
 - Release build: development bypasses are disabled and data-handling disclosures match the actual app and SDKs. Record unresolved risks and seek an independent review before collecting real users' account data.

Security sources

The additions above adapt primary-source guidance into Bodios-specific requirements. Security appendix retained and adapted for iOS on 26 September 2026; React Native security guidance rechecked. Recheck version-specific APIs during implementation.

- [S1] React Native: Security - <https://reactnative.dev/docs/security>
- [S2] OWASP: Authentication Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [S3] OWASP: Forgot Password Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html
- [S4] OWASP: Authorization Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

9 - TestFlight and App Store preparation

The target is an iPhone app distributed through Apple's App Store. Development completion, beta testing, and store approval are separate milestones. Do not promise approval or an exact launch date. The owner handles enrollment/payment and any required identity verification. Check current membership pricing and submission SDK requirements when preparing the release. [I3, I4]

COPY-PASTE AGENT PROMPT

Implement Stage 9 after the core flow passes Stage 7. Inspect the Xcode bundle identifier, signing team, supported iOS deployment target, version/build numbers, app icon, launch appearance and Release configuration. Check current Apple submission requirements and dependency compatibility instead of hardcoding an old Xcode/SDK requirement. Build and validate a signed archive when credentials are available. Do not commit certificates, signing credentials or private keys.

Prepare App Store Connect metadata: app name, description, support URL, privacy-policy URL, actual iPhone screenshots, age rating, export-compliance answers and accurate App Privacy disclosures based on actual code and SDK behavior. Add an accessible privacy-policy link inside the app. Inspect privacy manifests, required-reason API declarations and applicable SDK signatures; resolve missing declarations using actual API usage and vendor guidance, not invented reasons. Do not claim no data is collected without checking analytics and backend behavior. [I4, I5]

If account creation is enabled, implement account deletion inside the app. Review guideline 4.8 when adding Google or another third-party login, including whether an equivalent privacy-preserving option such as Sign in with Apple is needed. An exclusively first-party email/password system is not automatically required to add Apple login. Supply review access when login gates features and keep the backend available during review. [I4]

Prepare a TestFlight testing checklist covering install/update, offline logging, resume after force quit, videos, migration from an earlier build, and accounts if enabled. Explain any external-testing review requirement. Build all reviewable release materials first, then get explicit authorization before uploading to App Store Connect, inviting testers or submitting for App Review. Report exactly what is built locally, what is uploaded and what remains blocked. Release preparation alone is not authorization to publish.

Verified iOS references

Platform guidance checked 26 September 2026. Recheck at implementation and submission time. Library-specific references earlier in the document must be checked against the chosen package versions.

- [I1] React Native environment setup: <https://reactnative.dev/docs/set-up-your-environment>
- [I2] React Native Simulator: <https://reactnative.dev/docs/running-on-simulator-ios>
- [I3] Apple membership comparison: <https://developer.apple.com/support/compare-memberships/>
- [I4] Apple App Review Guidelines: <https://developer.apple.com/app-store/review/guidelines/>
- [I5] Apple third-party SDK requirements:
<https://developer.apple.com/support/third-party-SDK-requirements/>
- Apple TestFlight workflow: <https://help.apple.com/xcode/mac/current/en.lproj/dev2539d985f.html>